PlantDescriptionEngine

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-PlantDescriptionEngine-java-spring. 4 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-PlantDescriptionEngine-go.

PlantDescriptionEngine - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/PlantDescriptionEngine. Entry point: ./cmd/plantdescriptionengine. Module: arrowhead/ah5-support-go/plantdescriptionengine. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8432 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8432"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
PlantDescriptionEngine	Topology/include composition, named/metadata resolution, periodic alarms, DSO snapshot synchronization and monitor enrichment	Java state import, complete normalization/error parity and production qualification

Detailed implementation evidence

1 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-PlantDescriptionEngine-rust.

PlantDescriptionEngine (Rust)

Arrowhead 5.2.2 PlantDescriptionEngine support system. Entry point src/main.rs is built as binary plantdescriptionengine by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/plantdescriptionengine.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/PlantDescriptionEngine, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin plantdescriptionengine -- <config.json>

Listening port

Default 127.0.0.1:8432 (Arrowhead default, role plantdescriptionengine). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
PlantDescriptionEngine	Topology CRUD/include composition, named/metadata matching, periodic registration/mismatch alarms, atomic DSO snapshot synchronization, alarm acknowledgment	Monitoring-staleness alarms, full Java DTO normalization/state import and production qualification

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence

explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-PlantDescriptionEngine-java-spring/documentation/v5.2.2/PlantDescriptionEngine-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-PlantDescriptionEngine-java-spring/documentation/v5.2.2/PlantDescriptionEngine-v5.2.2-plantDescriptionManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-PlantDescriptionEngine-java-spring/documentation/v5.2.2/PlantDescriptionEngine-v5.2.2-plantDescriptionMonitor-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-PlantDescriptionEngine-java-spring/documentation/v5.2.2/PlantDescriptionEngine-v5.2.2-sysMonitor-AMQP-IDD.tex

- Go: Eclipse-Arrowhead-go/ah5-PlantDescriptionEngine-go/documentation/PlantDescriptionEngine/PlantDescriptionEngine-HTTPS-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/PlantDescriptionEngine/PlantDescriptionEngine-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the PlantDescriptionEngine system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the PlantDescriptionEngine system, which provides general management of the PlantDescriptionEngine system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `plantdescriptionengine`, port 8432 and base path `/plantdescriptionengine/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /plantdescriptionengine/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /plantdescriptionengine/general/mgmt/logs HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /plantdescriptionengine/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (PlantDescriptionEngine),” Eclipse Arrowhead, SD 5.2.2, 2026, plantDescriptionEngine-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-23	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

plantDescriptionManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"plantDescriptionManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `plantDescriptionManagement` service of the PlantDescriptionEngine system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4
2.2 operation get . . . 5
2.3 operation create . . . 6
2.4 operation replace . . . 7
2.5 operation update . . . 9
2.6 operation remove . . . 10

3 Data Models **12**
3.1 struct Connection . . . 12
3.2 struct ErrorMessage . . . 12
3.3 struct PdeSystem . . . 12
3.4 struct PlantDescriptionListResponse . . . 12
3.5 struct PlantDescriptionRequest . . . 12
3.6 struct PlantDescriptionResponse . . . 13
3.7 struct PlantDescriptionUpdate . . . 13
3.8 struct Port . . . 13
3.9 struct SystemPort . . . 13
3.10 Primitives . . . 14

4 References **15**

5 Revision History **16**
5.1 Amendments . . . 16
5.2 Quality Assurance . . . 16

1 Overview

This document describes the `plantDescriptionManagement` service interface of the PlantDescriptionEngine system, which provides management of plant descriptions: the engineered systems of the plant, the services they consume and produce (ports), and the connections between ports.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `plantDescriptionManagement` service interface

This document provides the Interface Design Description IDD to the *plantDescriptionManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `plantDescriptionManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `plantdescriptionengine`, port 8432 and base path `/plantdescriptionengine/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	GET	`/plant-descriptions`
`get`	GET	`/plant-descriptions/{id}`
`create`	POST	`/plant-descriptions`
`replace`	PUT	`/plant-descriptions/{id}`
`update`	PATCH	`/plant-descriptions/{id}`
`remove`	DELETE	`/plant-descriptions/{id}`

2 Service Operations

The interfaces of the `plantDescriptionManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `plantDescriptionManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 GET /plantdescriptionengine/mgmt/plant-descriptions?page={page}&size={size}&direction={direction}&sort_field={sort_field}&active={active}

Operation: query
Output: PlantDescriptionListResponse

Returns a page of plant descriptions, optionally only the active one.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sort_field* (String, optional); *active* (Boolean, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/mgmt/plant-descriptions?page=<page>&size=<size>&direction=<direction>&
    sort_field=<sort_field>&active=<active> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An query invocation.

```
{
  "entries": [
    {
      "id": 42,
      "plantDescription": "Line 1",
      "active": true,
      "include": [
        1
      ],
      "systems": [
        {
          "systemId": "sensor",
          "systemName": "TemperatureSensor",
          "metadata": {
            "unit": "celsius"
          },
          "ports": [
            {}
          ]
        }
      ],
      "connections": [
        {
          "priority": 1
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
```

```
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An query response.

2.2 GET /plantdescriptionengine/mgmt/plant-descriptions/*{id}*

Operation: get
Input: String
Output: PlantDescriptionResponse

Returns a plant description.
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/mgmt/plant-descriptions/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get invocation.

```
{
  "id": 42,
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "priority": 1
    }
  ],
```

```
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 4: An get response.

2.3 POST /plantdescriptionengine/mgmt/plant-descriptions

Operation: create
Input: PlantDescriptionRequest
Output: PlantDescriptionResponse

Creates a plant description; if active, it replaces the active one and the rules are synchronized.
A successful call returns `201 Created` with the output in the body.

```
POST /plantdescriptionengine/mgmt/plant-descriptions HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "priority": 1
    }
  ]
}
```

Listing 5: An create invocation.

```
{
```

```
  "id": 42,
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "priority": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 6: An create response.

2.4 PUT /plantdescriptionengine/mgmt/plant-descriptions/*{id}*

Operation: replace
Input: PlantDescriptionRequest
Output: PlantDescriptionResponse

Replaces a plant description.
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
PUT /plantdescriptionengine/mgmt/plant-descriptions/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
```

```
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "priority": 1
    }
  ]
}
```

Listing 7: An replace invocation.

```
{
  "id": 42,
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
```

```
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "priority": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 8: An replace response.

2.5 PATCH /plantdescriptionengine/mgmt/plant-descriptions/*{id}*

Operation: update
Input: PlantDescriptionUpdate
Output: PlantDescriptionResponse

Changes the given fields of a plant description.
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
PATCH /plantdescriptionengine/mgmt/plant-descriptions/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
```

```
        "portName": "out"
      },
      "priority": 1
    }
  ]
}
```

Listing 9: An update invocation.

```
{
  "id": 42,
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ]
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "priority": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 10: An update response.

2.6 DELETE /plantdescriptionengine/mgmt/plant-descriptions/*{id}*

Operation: remove
Input: String

Removes a plant description and its rules.
Path parameter: *id*.
A successful call returns `204 No Content`.

```
DELETE /plantdescriptionengine/mgmt/plant-descriptions/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 11: An remove invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.10, which are used to represent things like hashes, identifiers and texts.

3.1 struct Connection

A connection from a consumer port to a producer port.

Object Field	Value Type	Description
"consumer"	SystemPort	The consumer port of the connection.
"producer"	SystemPort	The producer port of the connection.
"priority"	Number	Priority of the resulting orchestration rules (1 highest).

3.2 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.3 struct PdeSystem

A system of the plant.

Object Field	Value Type	Description
"systemId"	String	Identifier of the system within the plant description.
"systemName"	String	Name of the registered system (optional; metadata may identify it instead).
"metadata"	Object<String>	Metadata the registered system must have.
"ports"	Array<Port>	Services the system consumes or produces.

3.4 struct PlantDescriptionListResponse

List of plant descriptions.

Object Field	Value Type	Description
"entries"	Array<PlantDescriptionResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct PlantDescriptionRequest

A plant description to create or replace.

Object Field	Value Type	Description
"plantDescription"	String	Name of the plant description.
"active"	Boolean	Make this the active plant description.
"include"	Array<Number>	Identifiers of included plant descriptions.
"systems"	Array<PdeSystem>	Systems to create or update.
"connections"	Array<Connection>	Connections between consumer and producer ports.

3.6 struct PlantDescriptionResponse

A stored plant description.

Object Field	Value Type	Description
"id"	Number	Identifier of the plant description.
"plantDescription"	String	Name of the plant description.
"active"	Boolean	Whether this is the active plant description.
"include"	Array<Number>	Identifiers of included plant descriptions.
"systems"	Array<PdeSystem>	Systems to create or update.
"connections"	Array<Connection>	Connections between consumer and producer ports.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.7 struct PlantDescriptionUpdate

Fields of a plant description to change; absent fields are kept.

Object Field	Value Type	Description
"plantDescription"	String	Name of the plant description.
"active"	Boolean	Make this the active plant description.
"include"	Array<Number>	Identifiers of included plant descriptions.
"systems"	Array<PdeSystem>	Systems to create or update.
"connections"	Array<Connection>	Connections between consumer and producer ports.

3.8 struct Port

A service a plant system consumes or produces.

Object Field	Value Type	Description
"portName"	String	Name of the port within its system.
"serviceDefinition"	String	Service definition name.
"serviceInterface"	String	Required interface template of the port, if any.
"metadata"	Object<String>	Metadata the service instance must have.
"consumer"	Boolean	True for a consumed service, false for a produced one.

3.9 struct SystemPort

Reference to a port of a plant system.

Object Field	Value Type	Description
"systemId"	String	Identifier of the system within the plant description.
"portName"	String	Name of the port within its system.

3.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “plantDescriptionManagement – Service Description (PlantDescriptionEngine),” Eclipse Arrowhead, SD 5.2.2, 2026, plantDescriptionEngine-v5.2.2-plantDescriptionManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-23	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

plantDescriptionMonitor HTTP/TLS/JSON

Interface Design Description

Service ID: *"plantDescriptionMonitor"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `plantDescriptionMonitor` service of the PlantDescriptionEngine system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query-plant-descriptions . . . 4
2.2 operation get-plant-description . . . 5
2.3 operation query-alarms . . . 6
2.4 operation get-alarm . . . 6
2.5 operation acknowledge-alarm . . . 7

3 Data Models **8**
3.1 struct Connection . . . 8
3.2 struct ErrorMessage . . . 8
3.3 struct MonitorPlantDescriptionListResponse . . . 8
3.4 struct MonitorPlantDescriptionResponse . . . 8
3.5 struct MonitorSystem . . . 8
3.6 struct PdeAlarmListResponse . . . 9
3.7 struct PdeAlarmResponse . . . 9
3.8 struct PdeAlarmUpdate . . . 9
3.9 struct Port . . . 10
3.10 struct SystemPort . . . 10
3.11 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `plantDescriptionMonitor` service interface of the PlantDescriptionEngine system, which provides read access to the plant descriptions enriched with the matched registered systems and their monitoring state, and to the alarms raised when the running cloud differs from the description.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `plantDescriptionMonitor` service interface

This document provides the Interface Design Description IDD to the *plantDescriptionMonitor – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `plantDescriptionMonitor` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `plantdescriptionengine`, port 8432 and base path `/plantdescriptionengine/monitor`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query-plant-descriptions`	GET	`/plant-descriptions`
`get-plant-description`	GET	`/plant-descriptions/{id}`
`query-alarms`	GET	`/alarms`
`get-alarm`	GET	`/alarms/{id}`
`acknowledge-alarm`	PATCH	`/alarms/{id}`

2 Service Operations

The interfaces of the `plantDescriptionMonitor` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `plantDescriptionMonitor` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 GET /plantdescriptionengine/monitor/plant-descriptions?page=*{page}* &size=*{size}*&direction=*{direction}*&sort_field={sort_field}&active=*{active}*

Operation: query-plant-descriptions
Output: MonitorPlantDescriptionListResponse

Returns a page of plant descriptions with monitoring data.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sort_field* (String, optional); *active* (Boolean, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/monitor/plant-descriptions?page=<page>&size=<size>&direction=<direction
    >&sort_field=<sort_field>&active=<active> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An query-plant-descriptions invocation.

```
{
  "entries": [
    {
      "id": 42,
      "plantDescription": "Line 1",
      "active": true,
      "include": [
        1
      ],
      "systems": [
        {
          "systemId": "sensor",
          "systemName": "TemperatureSensor",
          "metadata": {
            "unit": "celsius"
          },
          "ports": [
            {}
          ],
          "registeredSystemName": "string",
          "lastObservationTime": "2026-10-22T12:00:00Z"
        }
      ],
      "connections": [
        {
          "priority": 1
        }
```

```
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An query-plant-descriptions response.

2.2 GET /plantdescriptionengine/monitor/plant-descriptions/*{id}*

Operation: get-plant-description
Input: String
Output: MonitorPlantDescriptionResponse

Returns a plant description with monitoring data.
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/monitor/plant-descriptions/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-plant-description invocation.

```
{
  "id": 42,
  "plantDescription": "Line 1",
  "active": true,
  "include": [
    1
  ],
  "systems": [
    {
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "metadata": {
        "unit": "celsius"
      },
      "ports": [
        {
          "portName": "out",
          "serviceDefinition": "temperatureReading",
          "serviceInterface": "string",
          "metadata": {
            "unit": "celsius"
          },
          "consumer": "TemperatureDisplay"
        }
      ],
      "registeredSystemName": "string",
      "lastObservationTime": "2026-10-22T12:00:00Z"
    }
  ],
  "connections": [
    {
      "consumer": {
        "systemId": "sensor",
        "portName": "out"
      },
      "producer": {
        "systemId": "sensor",
        "portName": "out"
```

```
      },
      "priority": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 4: An get-plant-description response.

2.3 GET /plantdescriptionengine/monitor/alarms?page={page}&size={size}&direction={direction}&sort_field={sort_field}&system_name={system_name}&severity={severity}&acknowledged={acknowledged}

Operation: query-alarms
Output: PdeAlarmListResponse

Returns a page of alarms, filtered by system name, severity and acknowledgement.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sort_field* (String, optional); *system_name* (String, optional); *severity* (String, optional); *acknowledged* (Boolean, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/monitor/alarms?page=<page>&size=<size>&direction=<direction>&sort_field=
    <sort_field>&system_name=<system_name>&severity=<severity>&acknowledged=<acknowledged> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An query-alarms invocation.

```
{
  "entries": [
    {
      "id": 42,
      "systemId": "sensor",
      "systemName": "TemperatureSensor",
      "acknowledged": true,
      "severity": "INFO",
      "cause": "SYSTEM_NOT_REGISTERED",
      "description": "Display may read",
      "raisedAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z",
      "clearedAt": "2026-10-22T12:00:00Z",
      "acknowledgedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 6: An query-alarms response.

2.4 GET /plantdescriptionengine/monitor/alarms/{id}

Operation: get-alarm
Input: String
Output: PdeAlarmResponse

Returns an alarm.

Path parameter: *id*.

A successful call returns `200 OK` with the output in the body.

```
GET /plantdescriptionengine/monitor/alarms/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 7: An get-alarm invocation.


```
{
  "id": 42,
  "systemId": "sensor",
  "systemName": "TemperatureSensor",
  "acknowledged": true,
  "severity": "INFO",
  "cause": "SYSTEM_NOT_REGISTERED",
  "description": "Display may read",
  "raisedAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z",
  "clearedAt": "2026-10-22T12:00:00Z",
  "acknowledgedAt": "2026-10-22T12:00:00Z"
}
```


Listing 8: An get-alarm response.

2.5 PATCH /plantdescriptionengine/monitor/alarms/*{id}*

Operation: acknowledge-alarm
Input: PdeAlarmUpdate
Output: PdeAlarmResponse

Acknowledges an alarm.
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
PATCH /plantdescriptionengine/monitor/alarms/<id> HTTP/1.1
Host: plantdescriptionengine:8432
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "acknowledged": true
}
```

Listing 9: An acknowledge-alarm invocation.


```
{
  "id": 42,
  "systemId": "sensor",
  "systemName": "TemperatureSensor",
  "acknowledged": true,
  "severity": "INFO",
  "cause": "SYSTEM_NOT_REGISTERED",
  "description": "Display may read",
  "raisedAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z",
  "clearedAt": "2026-10-22T12:00:00Z",
  "acknowledgedAt": "2026-10-22T12:00:00Z"
}
```


Listing 10: An acknowledge-alarm response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.11, which are used to represent things like hashes, identifiers and texts.

3.1 struct Connection

A connection from a consumer port to a producer port.

Object Field	Value Type	Description
"consumer"	SystemPort	The consumer port of the connection.
"producer"	SystemPort	The producer port of the connection.
"priority"	Number	Priority of the resulting orchestration rules (1 highest).

3.2 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.3 struct MonitorPlantDescriptionListResponse

List of plant descriptions with monitoring data.

Object Field	Value Type	Description
"entries"	Array<MonitorPlantDescription Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.4 struct MonitorPlantDescriptionResponse

A plant description with monitoring data.

Object Field	Value Type	Description
"id"	Number	Identifier of the plant description.
"plantDescription"	String	Name of the plant description.
"active"	Boolean	Whether the entry is in force.
"include"	Array<Number>	Identifiers of included plant descriptions.
"systems"	Array<MonitorSystem>	Systems to create or update.
"connections"	Array<Connection>	Connections between consumer and producer ports.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.5 struct MonitorSystem

A plant system with its matched registered system and monitoring data.

Object Field	Value Type	Description
"systemId"	String	Identifier of the system within the plant description.
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"metadata"	Object<String>	Metadata the registered system must have.
"ports"	Array<Port>	Services the system consumes or produces.
"registeredSystem Name"	String	Name of the registered system matched to this plant system.
"monitoring"	Object	Latest monitoring information from SysMonitor.
"lastObservationTime"	DateTime	Time of the latest stored observation of the system.

3.6 struct PdeAlarmListResponse

List of alarms.

Object Field	Value Type	Description
"entries"	Array<PdeAlarmResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct PdeAlarmResponse

An alarm.

Object Field	Value Type	Description
"id"	Number	Identifier of the alarm.
"systemId"	String	Identifier of the system within the plant description.
"systemName"	String	Name of the system the alarm concerns.
"acknowledged"	Boolean	Whether an operator acknowledged the alarm.
"severity"	String	Severity of the alarm.
"cause"	String	Alarm cause: `SYSTEM_NOT_REGISTERED`, `MULTIPLE_MATCHES`, `SYSTEM_NOT_IN_DESCRIPTION`, `NOT_MONITORED` or `NO_RECENT_OBSERVATION`.
"description"	String	Human-readable alarm text.
"raisedAt"	DateTime	Time the alarm was raised.
"updatedAt"	DateTime	UTC time of the last change.
"clearedAt"	DateTime	Time the alarm was cleared, if cleared.
"acknowledgedAt"	DateTime	Time of the acknowledgement.

3.8 struct PdeAlarmUpdate

Acknowledgement of an alarm.

Object Field	Value Type	Description
"acknowledged"	Boolean	Set to true to acknowledge the alarm.

3.9 struct Port

A service a plant system consumes or produces.

Object Field	Value Type	Description
"portName"	String	Name of the port within its system.
"serviceDefinition"	String	Service definition name.
"serviceInterface"	String	Required interface template of the port, if any.
"metadata"	Object<String>	Metadata the service instance must have.
"consumer"	Boolean	True for a consumed service, false for a produced one.

3.10 struct SystemPort

Reference to a port of a plant system.

Object Field	Value Type	Description
"systemId"	String	Identifier of the system within the plant description.
"portName"	String	Name of the port within its system.

3.11 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “plantDescriptionMonitor – Service Description (PlantDescriptionEngine),” Eclipse Arrowhead, SD 5.2.2, 2026, plantDescriptionEngine-v5.2.2-plantDescriptionMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-23	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the PlantDescriptionEngine system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation echo . . . 4
2.2 operation power . . . 4
2.3 operation status . . . 4

3 Data Models **6**
3.1 struct Observation . . . 6
3.2 Primitives . . . 6

4 References **7**

5 Revision History **8**
5.1 Amendments . . . 8
5.2 Quality Assurance . . . 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of PlantDescriptionEngine.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, PlantDescriptionEngine registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.PlantDescriptionEngine.PlantDescriptionEngine",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher is the `SysMonitorPublisher` of the common library, configured with `sysmonitor.broker.address`, `sysmonitor.broker.port`, `sysmonitor.local.cloud` (default: from the system certificate), `sysmonitor.publish.interval` and `sysmonitor.ssl.*` (default: the `server.ssl` stores). The publisher connects to the broker (default `amqp-broker`, port 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.PlantDescriptionEngine.PlantDescriptionEngine.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.PlantDescriptionEngine.PlantDescriptionEngine.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.PlantDescriptionEngine.PlantDescriptionEngine.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "PlantDescriptionEngine-echo-1792670400000-42",
  "providerSystem": "PlantDescriptionEngine",
  "serviceInstance": "PlantDescriptionEngine",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "PlantDescriptionEngine is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
```

```
17     }
18 }
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power`, `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (PlantDescriptionEngine),” Eclipse Arrowhead, SD 5.2.2, 2026, plantDescriptionEngine-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-23	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

HTTPS, mandatory mutual TLS, Arrowhead identity-token header and JSON request/response documents. General management is sysop/whitelist protected; enrollment endpoints use approved certificate fingerprints and profile/serial checks instead of ordinary identity tokens. Error responses contain errorMessage, errorCode, origin and timestamp. Exact DTO and status/error parity is not certified.

1.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/plantdescriptionengine/mgmt/sync-status	management
GET	/plantdescriptionengine/mgmt/plant-descriptions	management
GET	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
POST	/plantdescriptionengine/mgmt/plant-descriptions	management
PUT	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
PATCH	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
DELETE	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
GET	/plantdescriptionengine/monitor/plant-descriptions	identity
GET	/plantdescriptionengine/monitor/plant-descriptions/{id}	identity
GET	/plantdescriptionengine/monitor/alarms	identity
GET	/plantdescriptionengine/monitor/alarms/{id}	identity
PATCH	/plantdescriptionengine/monitor/alarms/{id}	identity
GET	/plantdescriptionengine/monitor/echo	certificate
GET	/plantdescriptionengine/general/mgmt/get-config	management
POST	/plantdescriptionengine/general/mgmt/logs	management

1.2 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Topology/include composition, named and metadata matching, periodic alarms, and atomic PDE-to-DSO snapshot synchronization are implemented. Missing producers retain managed empty scopes; nonnegative priorities rank eligible providers. Java SQL import, complete DTO normalization and production qualification remain outside this extension.

1 PlantDescriptionEngine

1.1 System overview

Native Rust development profile for local Java source `PlantDescriptionEngine/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. Configured CRLs and TLS material reload without process restart; verification fails closed when required material is unavailable. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Topology CRUD/include composition, named/metadata matching, periodic registration/mismatch alarms, atomic DSO snapshot synchronization, alarm acknowledgment

1.5 Open release gates

Monitoring-staleness alarms, full Java DTO normalization/state import and production qualification. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `echo`, `generalManagement`, `plantDescriptionManagement`, `plantDescriptionMonitor`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. Optional native AMQP monitoring publishes echo, power and status observations.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/plantdescriptionengine/general/mgmt/get-config	management
POST	/plantdescriptionengine/general/mgmt/logs	management
GET	/plantdescriptionengine/monitor/echo	certificate
GET	/plantdescriptionengine/mgmt/sync-status	management
GET	/plantdescriptionengine/mgmt/plant-descriptions	management
POST	/plantdescriptionengine/mgmt/plant-descriptions	management
GET	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
PUT	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
PATCH	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
DELETE	/plantdescriptionengine/mgmt/plant-descriptions/{id}	management
GET	/plantdescriptionengine/monitor/plant-descriptions	identity
GET	/plantdescriptionengine/monitor/plant-descriptions/{id}	identity
GET	/plantdescriptionengine/monitor/alarms	identity
GET	/plantdescriptionengine/monitor/alarms/{id}	identity
PATCH	/plantdescriptionengine/monitor/alarms/{id}	identity

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.

3 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains `serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.